



with quotient filters are there still false positives

Yes, quotient filters still produce false positives.  systemdesign.one

Why False Positives Happen

- **Probabilistic Nature:** Like Bloom filters, a quotient filter is a space-efficient probabilistic data structure. It trades off absolute accuracy for compact memory use.  ACM Digital Library +2
- **Fingerprint Collisions:** A quotient filter stores a truncated hash fingerprint (divided into a quotient and a remainder) instead of the full key. If a queried item that is *not* in the set happens to generate a remainder and quotient mapping that collides with an existing entry, the filter will incorrectly return a "yes" (a false positive).  Khoury College of Co... +3
- **No False Negatives:** If the filter says an item is **not** in the set, it is definitively absent. False positives only apply when the filter claims an item **is** present.  Williams College +2

Controlling the False Positive Rate

- **Bits Allocation:** You can lower the probability of false positives by increasing the number of bits used for the remainder portion of the fingerprint.  batuhanaydin.dev
- **Space Trade-off:** Reducing the false positive rate requires more storage bits per item, increasing the overall memory footprint of the filter.  batuhanaydin.dev

If you'd like, I can explain:

- How the **quotient and remainder** are calculated and stored
- How quotient filter collision resolution compares to **Bloom filters**

Ask anything





why/how do quotient filters allow removals

Quotient filters allow removals because they store exact remainder fingerprints and use a hardcoded collision resolution scheme that preserves the metadata of clustered elements.

Unlike a standard Bloom filter, which destroys structural information by bitwise-ORing multiple hashes into a single array, a quotient filter allocates a specific slot for each remainder. When collisions occur, it shifts the remainders into adjacent slots using a form of open addressing (linear probing).

Because the exact remainders are preserved, you can cleanly delete an item by reversing this process.

How the Removal Process Works

To remove an item, the filter performs the following steps:

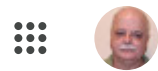
1. Find the item's remainder and its slot in the filter.

2. Verify the item's fingerprint matches the one stored in the slot.

3. If the fingerprints match, clear the slot to mark the item for removal.

4. If the fingerprints do not match, the item was not in the filter.





3. **Update Metadata Bits:** Each slot in a quotient filter tracks metadata using 3 bits (typically called *is_occupied*, *is_continuation*, and *is_shifted*). During a removal, these bits are updated for the shifted items so the filter still knows exactly which quotient each remainder originally belonged to.

Comparison: Why Bloom Filters Can't Do This

Feature	Quotient Filter	Standard Bloom Filter
Storage Method	Stores distinct, individual remainder fingerprints in an array.	Sets multiple independent bits across an array to <code>1</code> .
Collision Handling	Shifts remainders into adjacent slots while tracking their origin with metadata bits.	Overwrites bits. Multiple different items will set the exact same bits to <code>1</code> .
Removal Mechanism	Clears the remainder and shifts neighboring elements back into place.	Impossible. Clearing a bit to delete one item would accidentally delete parts of other items sharing that bit.

*Note: While **Counting Bloom Filters** allow deletions by replacing bits with counters, they require significantly more memory (usually 3 to 4 times more space) than a quotient filter to achieve the same accuracy.*



+





1. Hash Collisions (The Mathematical Reality)

Because a hash function maps an infinite set of possible inputs to a fixed-size integer, **hash collisions are mathematically guaranteed to exist** (by the Pigeonhole Principle).

If two distinct values happen to hash to the exact same integer, they will produce the exact same quotient and remainder. To the quotient filter, these two items are completely indistinguishable.

- **How it affects operations:** If `Item A` is inserted, and `Item B` hashes to the exact same value, looking up `Item B` will return a false positive.
- **Why it's acceptable:** In practice, quotient filters use strong, uniform cryptographic or non-cryptographic hash functions (like MurmurHash3 or SHA-256) with large outputs (e.g., 64 bits). The mathematical probability of two specific items colliding on a 64-bit hash is 1 in 2^{64} (about 1 in 18 quintillion), making it rare enough to ignore for standard applications.

2. Fingerprint Collisions (Inside the Filter)

Even if two distinct values have completely different 64-bit hashes, they can still collide inside the filter. This is because the filter intentionally **truncates** the hash to save space.

A quotient filter breaks the hash into:

- **The Quotient (q bits):** Used as the index address in the table.
- **The Remainder (r bits):** The actual "fingerprint" stored in the slot.

If two different values have hashes that share the exact same q and r bits—even if the rest of their 64-bit hashes are different—they will result in a **fingerprint collision**. This is the primary driver of the filter's false positive rate, which is



**Fingerprint Collision**

Two items have different hashes, but share the same quotient and remainder.

The filter detects a match during a lookup, causing a standard false positive.

Slot Collision

Two items have the **same quotient** but **different remainders**.

The filter handles this perfectly. It shifts the second remainder to the next available slot and uses metadata bits to keep track of it.

Would you like to explore **how the 3 metadata bits resolve slot collisions**, or should we look at the mathematical formula used to calculate the **optimal remainder size** for a target false positive rate?



with 32 bit hashes, how many values should we process so probability of a collision is 50%

To achieve a **50% probability of a collision** with 32-bit hashes, you need to process approximately **77,163 values**.

This surprisingly small number is a result of the **Birthday Paradox**. It demonstrates that you do not need anywhere near 4.29 billion values (the total capacity of a 32-bit integer) to encounter a hash collision. Instead, the number of pairs grows quadratically with the number of values processed, causing the likelihood of a collision to climb very quickly.

The Mathematics Behind It

The approximation formula derived from the birthday problem to find the number





$$n \approx \sqrt{5,954,342,217} \approx 77,162.74$$

What This Means for Quotient Filters

Because 77,163 values is a relatively small dataset for modern applications, using a raw **32-bit hash function** makes the filter highly vulnerable to early hash collisions. This is why practical implementations of quotient filters and Bloom filters almost always use **64-bit hashes** (where a 50% collision probability requires roughly **5.06 billion values**) or even 128-bit hashes.

Would you like to calculate how many values it would take to hit a 50% collision probability using a **64-bit hash**, or should we look at how a **higher collision rate** affects the filter's memory performance?

Show Code <>



and with 64 bit hashes?

To achieve a 50% probability of a collision with 64-bit hashes, you need to process approximately 5,060,000,000 values (5.06 billion values).

The Math for 64-Bit

Using the exact same Birthday Paradox formula, where $N = 2^{64}$ (approximately 18.45 quintillion possible hash buckets):

$$n \approx \sqrt{2 \times 2^{64} \times \ln(2)}$$





While 77,163 values (for 32-bit) can easily be reached in seconds by almost any modern application, **5.06 billion values** is larger than the dataset size of most in-memory filter deployments.

If your application processes fewer than a few hundred million unique items over its lifetime, a 64-bit hash function guarantees that the probability of a raw hash collision is practically negligible.

Would you like to see how this **5-billion-item threshold** scales if you change the acceptable **collision probability to 1% or 0.1%**, or would you like to calculate how much **RAM** a quotient filter would need to hold that many elements?



how many bits should we use for quotient filters

To determine how many bits a quotient filter needs, you must **calculate the size of the quotient (q) and the size of the remainder (r) separately**.

As a general rule of thumb, a well-optimized quotient filter typically requires **between 10 and 15 total bits per element** to maintain a low false positive rate (around 1%) while operating efficiently.

The exact number of bits depends entirely on two variables: **how many items you want to store (n)** and your **target false positive rate (ϵ)**.

1. Calculating the Quotient Bits (q)

The quotient bits determine the number of slots in the filter's main array (where





(Where α is the load factor, usually chosen to be around **0.75**).

2. Calculating the Remainder Bits (r)

The remainder bits act as the fingerprint that uniquely identifies the item within its slot cluster. This is what directly controls your false positive rate.

- **The Formula:** To achieve a target false positive rate of ϵ , the number of remainder bits should be:

$$r \approx \log_2 \left(\frac{1}{\epsilon} \right)$$

3. Adding the Metadata Bits

Every single slot in a quotient filter requires exactly **3 metadata bits** (*is_occupied*, *is_continuation*, and *is_shifted*) to handle collisions and allow for removals.

Quick Reference Chart (At 75% Load Factor)

Here is how many total bits you need per item based on your desired false positive rate:

Target False Positive Rate (ϵ)	Remainder Bits (r)	Metadata Bits	Approx. Total Bits Per Item
6.25% (1 in 16)	4 bits	3 bits	~9.3 bits
1.00% (1 in 100)	7 bits	3 bits	~13.3 bits
0.10% (1 in 1,000)	10 bits	3 bits	~17.3 bits





1. **Slots needed:** $1,000,000 / 0.75 = 1,333,333$. The next power of two is $2^{21} = 2,097,152$, so **q = 21**.
2. **Remainder size:** For a 1% rate, **r = 7** bits.
3. **Total size per slot:** 7 remainder bits + 3 metadata bits = **10 bits per slot**.
4. **Total filter memory:** $2,097,152 \text{ slots} \times 10 \text{ bits} \approx 21 \text{ Megabits}$ (or **2.62 Megabytes**).

Would you like to **plug in your specific dataset size and target error rate** to calculate the exact memory footprint you'll need, or would you like to compare this space efficiency directly against a **Bloom filter**?



with this concrete example, how does the birthday paradox numbers work out

In this concrete example, the Birthday Paradox numbers work out **so that a fingerprint collision inside the filter is 100% guaranteed**, which is exactly how a quotient filter achieves its 1% false positive rate.

Here is how the math breaks down when you insert those 1 million items:

1. The Raw Hash (No Collisions)

First, you hash your 1 million items using a **64-bit hash function**.

- As we calculated earlier, you need about 5.06 billion items to have a 50% chance of a raw hash collision.
- With only 1 million items, the chance of two items producing the exact same 64-





3. Applying the Birthday Paradox

According to the Birthday Paradox formula, how many items do you need to process before two items accidentally share the exact same 28-bit identity?

$$n \approx \sqrt{2 \times 268,435,456 \times \ln(2)} \approx 19,295 \text{ items}$$

You only need **19,295 items** to have a 50% chance of a fingerprint collision. Because you are inserting **1,000,000 items**—well past that threshold—the mathematical probability of having *at least one* fingerprint collision is effectively **100%**.

In fact, you will have thousands of them.

Why This Doesn't Break the Filter

This sounds alarming, but it is precisely how the filter is engineered to work.

- **Slot Collisions (Different Remainders):** If two items share the same 21-bit quotient but have *different* 7-bit remainders, the filter safely resolves this. It shifts one of them to the next slot and uses metadata bits to remember its original home. **Zero false positives are caused here.**
- **Fingerprint Collisions (Same Remainder):** A false positive only occurs if a brand-new item shares both the exact same quotient *and* the exact same remainder as an item already in the filter.

Because the remaining 7 bits are uniformly distributed, any random item you look up has a 1 in 2^7 chance of hitting a matching remainder in that slot. Since $1/128 \approx 0.0078$, your **false positive rate lands right around 0.78% (just under your 1% target)**.

Would you like to see how the filter uses its **3 metadata bits** to tell the difference between elements that suffered a slot collision, or should we look at how

.....





How many full coincidences?

When inserting **1,000,000 items** into a 28-bit fingerprint space ($2^{28} = 268,435,456$ combinations), you will experience an expected average of **1,860 full coincidences** (fingerprint collisions).

A "full coincidence" means that an item being inserted generates the *exact same* 21-bit quotient and 7-bit remainder as an item that is already stored in the filter.

What This Means for Your Data

- **1,860 items** will completely vanish structurally into the filter because they map onto an existing identical fingerprint.
- If you query any of these 1,860 items, the filter will say "Yes, I've seen this" (technically a true positive, because you did insert it).
- If you try to **delete** one of these 1,860 items, the filter will successfully delete the fingerprint slot—but it will inadvertently delete *both* items that shared it.

Why This Number is Safe

While 1,860 sounds like a lot of collisions, remember that it is distributed across a massive 1-million-item dataset. It represents less than **0.19%** of your entire collection.

Because the background "noise" of occupied slots is so low compared to the 268 million possible slots, a completely random, non-existent item being queried still only has about a **0.78% (1 in 128) chance** of accidentally guessing one of those filled slots.

If you want to completely eliminate these 1,860 deletions errors or lower the false positive rate even further, I can show you how adding just **2 more bits to the remainder** changes these numbers. Would you like to **see that calculation**, or should we look at how the filter's **performance holds up** as it gets closer to being completely full?





so, how much more powerful are quotient filters than bloom filters

It is more accurate to call quotient filters highly functional rather than strictly more "powerful" than Bloom filters. When it comes to raw space efficiency, they are neck-and-neck, but quotient filters offer **massive structural and operational advantages** for real-world software engineering.

The direct comparison below shows where quotient filters outclass Bloom filters, and where they fall short.

Where Quotient Filters Win

- **Support for Deletions:** As discussed earlier, a quotient filter natively supports item removals because it preserves exact fingerprint data and resolves collisions cleanly. A standard Bloom filter cannot delete items without risking corrupting other data.
- **Hardware Cache Locality:** This is their biggest performance advantage. A Bloom filter requires hashing an item k times, which jumps to k completely random memory locations across a massive array. This forces costly CPU cache misses. A quotient filter hashes an item *once* and scans a single, continuous block of memory slots (a local cluster), making it incredibly fast and **cache-friendly**.
- **Dynamic Resizing:** If a quotient filter gets too full (approaching 80–85% capacity), it can be doubled in size in memory. You simply iterate through the slots, shift bits over, and re-allocate them. A standard Bloom filter cannot be resized without throwing it away and completely rebuilding it from the raw source data.
- **SSD/Disk Friendliness:** Because they access memory in continuous blocks rather than random scattered bits, quotient filters can be efficiently paged to disk or flash storage.





degrades smoothly as you fill it up, but it never slows down. A quotient filter becomes painfully slow if it goes past 85% capacity because collision chains grow long, forcing the CPU to scan down a massive line of shifted slots.

- **Simpler Implementation:** Bloom filters are trivial to implement in a few lines of code. Quotient filters require complex bit-shifting math and tracking state across three distinct metadata bits.

Direct Comparison Summary

Feature	Quotient Filter	Bloom Filter
Primary Advantage	Deletions, cache locality, resizing	Raw space efficiency, simplicity
CPU Cache Hits	Excellent (scans continuous slots)	Poor (jumps randomly across memory)
Item Deletions	Yes (native)	No (requires Counting Bloom variation)
Can be Resized?	Yes	No
When it Fills Up	Becomes very slow	False-positive rate climbs, but stays fast

The Verdict

If you are operating strictly in **RAM** and your dataset is **read-only/static**, a **Bloom filter** is still a highly optimal choice.

However, if you are designing a high-performance system that requires **deletions**,

+

🎤





but you still have no 100% certainty that deletions work correctly, right? you may be deleting a certain string and oth... ▾

You are **100% correct**. This is a critical, subtle vulnerability of probabilistic filters that many engineers overlook.

If you attempt to delete `String A`, and it happens to share a **full coincidence** (the exact same quotient and remainder) with a completely different `String B` that is also in the filter, **deleting `String A` will accidentally delete `String B` as well.**

From that moment forward, if you query the filter for `String B`, it will return `False` (a false negative), breaking the absolute rule that probabilistic filters should never have false negatives.

How Different Filters Handle This Risk

Because of this exact limitation, a quotient filter (and similarly, a Cuckoo filter) cannot safely be treated as a definitive master record for a dynamic database. Instead, production systems handle this constraint in two ways:

1. The Filter is Backed by Real Storage (Standard Architecture)

In databases like RocksDB or Cassandra, a quotient filter is only used as a fast "gatekeeper" in front of actual storage (like an SSD or hard drive).

- When you delete an item, you don't blindly delete it from the filter alone.
- The system checks the actual disk to verify the string exists.
- If a full coincidence happens, the system knows it cannot simply erase the





As we calculated earlier, with a 28-bit fingerprint, you get **1,860 accidental double-deletions**. But look what happens if you upgrade to a **64-bit fingerprint space** while keeping your 1 million items:

- Available states: $2^{64} \approx 18.4$ quintillion
- Number of items: 1, 000, 000
- Expected full coincidences: **0.000000000027**

At 64 bits, the chance of a collision causing a corrupt deletion is so low that a cosmic ray or hardware RAM failure is trillions of times more likely to corrupt your data first.

The Bottom Line

You are entirely right to be skeptical. **A quotient filter cannot guarantee 100% correct deletions on its own** if the dataset size approaches the mathematical limits of its fingerprint size. It is a probabilistic structure, meaning its deletions are inherently probabilistic too.

Are you evaluating this for a system where **data corruption/false negatives are completely unacceptable**, or are you designing a **cache layer** where a rare missed item can just be fetched from a slower database? Let me know the **architecture of your project** so we can look at the safest approach.



how does backing up with real storage solve this case: strings

A and B map to the same hash, so same quotient and same ... ▾

You have perfectly diagnosed the trap. If you blindly delete the fingerprint from the





Here are the two ways databases use real storage to solve this case:

Method 1: The "Leave It Alone" Strategy (Tombstones)

Modern databases rarely delete data inline because modifying files on disk is slow. Instead, when you delete `String A`, the database writes a **Tombstone** (a marker that says "`String A` is deleted") to the real disk storage.

1. **The Deletion:** You delete `String A`. The database leaves the quotient filter completely untouched. The fingerprint for `A` and `B` stays in the filter.
2. **Querying `String B`:** You look up `String B`. The filter says **"YES"** (because the fingerprint is still there). The database then checks the real disk, sees `String B` actually exists, and returns it. (Performance is fast).
3. **Querying `String A`:** You look up `String A`. The filter says **"YES"** (because the fingerprint is still there). The database checks the real disk, encounters the **Tombstone** for `A`, and correctly tells the user "No, `A` does not exist."

Result: Leaving the filter alone causes a harmless false positive for `String A`, but it completely protects `String B` from being falsely reported as missing.

Method 2: The "Read-Before-Write" Verification

If the system *must* clear memory immediately, it forces a quick read from the real storage before modifying the filter bits:

1. **The Deletion Command:** You tell the system to delete `String A`.
2. **The Disk Check:** The system looks at the quotient filter, finds the fingerprint, and notes its exact position. Before touching it, the system queries the real storage: *"Are there any other active strings in the database that map to this exact filter position?"*
3. **The Discovery:** The real storage looks at its indexes and says, "Yes, `String B` is still active and maps to that exact same fingerprint."





hours, the database merges its disk files and purges deleted data. During this process, it throws away the old, messy quotient filter and **builds a brand-new one from scratch** using only the surviving data currently on disk.

When the new filter is built, `String B` gets added, but since `String A` is long gone, the filter is perfectly clean again.

Does this **tombstone and compaction architecture** make sense for your system design, or are you working in an **environment without a persistent disk** (like a pure in-memory cache) where you can't rely on background rebuilds? Let me know your **hardware setup** so we can look at options.

